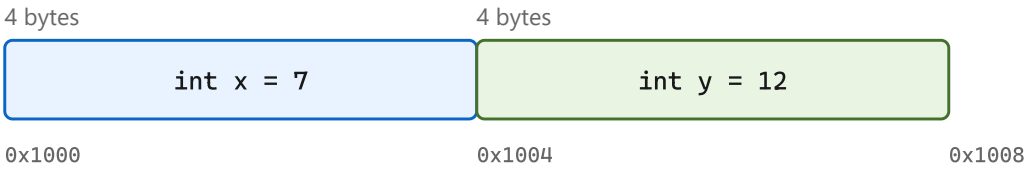
Pointers and References

Pointers, references and the memory they point at: where variables live, what an address is, arrays and decay, const, new and delete, and the bugs that follow. Assumes C++ Basics.

Memory is a row of numbered boxes

Your program's memory is a long row of numbered boxes, one per byte. The number of a box is its **address**. A variable is just a name for one or more boxes next to each other: an `int` takes 4 of them, and the address of the variable is the address of the first.



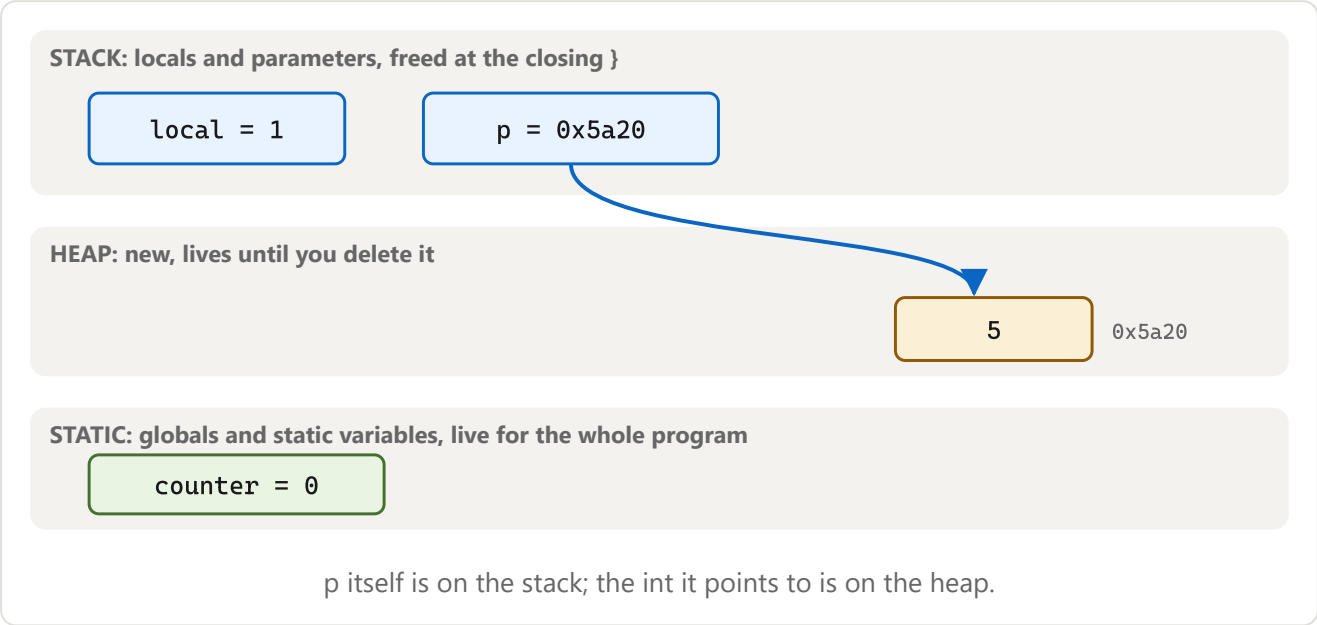
A variable is a name for some boxes; its address is where the first box is.

```
int x = 7;
int y = 12;
std::cout << &x << '\n';          // the address of x, such as 0x7ffc3a10
// 4: bytes used by an int on most machines
std::cout << sizeof(x) << '\n';
```

MADE-UP NUMBERS The addresses in this lesson are invented to keep the pictures readable. Real ones differ on every run.

Where variables live: stack, heap, static

Memory is split into regions with different lifetime rules. The rule decides where a variable should live.



p itself is on the stack; the int it points to is on the heap.

REGION	WHAT GOES THERE	LIFETIME	SPEED AND SIZE
Stack	local variables, function parameters	automatic: freed when the scope `{ }` ends	very fast, small (about 1 to 8 MB)
Heap (free store)	objects created with `new` or `malloc`	manual: lives until you `delete` it	slower, huge
Static	globals, `static` variables	the whole program	fixed at startup

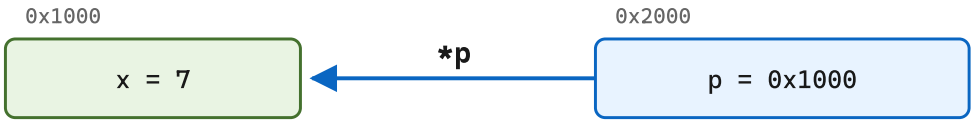
```
// static storage: exists for the whole program
int counter = 0;

void f() {
    // stack: created on entry, destroyed on exit
    int local = 1;
    // the int is on the HEAP; p itself is on the stack
    int* p = new int(5);
    delete p;                  // the heap int is freed here, by hand
}
```

MENTAL MODEL Stack: a pile of plates, popped automatically. Heap: a storage unit you rent until you hand the key back.

Pointers from scratch

A **pointer** is a variable whose value is an **address**. The type says what lives at that address, so a `int*` points to an `int`.



p stores an address; *p follows it to the value

The pointer is a separate variable. Its value is the address of x.

SYNTAX	MEANING
`int* p`	declare `p` as "pointer to int"
`&x`	address-of: the address of `x`
`*p`	dereference: the thing `p` points to
`p->m`	shorthand for `(*p).m` when `p` points to a struct
`nullptr`	the pointer that points to nothing

```
int x = 7;
int* p = &x;      // p holds the address of x
int y = *p;       // follow the arrow: y is 7 (a copy)
*p = 9;           // write through it: x is now 9
p = nullptr;      // points at nothing; never dereference it
```

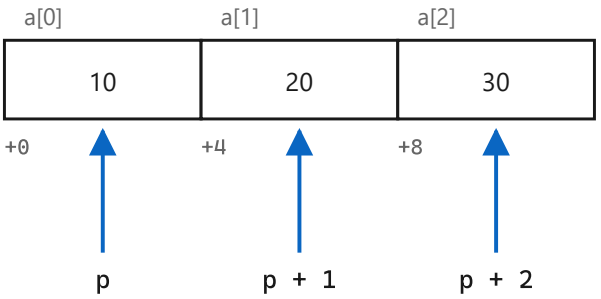
CAREFUL Dereferencing `nullptr`, an uninitialized pointer or a freed pointer is undefined behavior. C++ will not stop you.

WHAT A POINTER DOES NOT KNOW Only an address and a type: not **how many** elements are there, nor **who** frees them. That is the root of most pointer bugs.

KEY POINT "What is the difference between `NULL` and `nullptr`?" `NULL` is a macro for a null pointer constant, usually plain `0` (an integer), so `f(NULL)` can call `f(int)` instead of `f(char*)`. `nullptr` has its own type and only converts to pointers.

Pointers and arrays

An array is a row of elements next to each other. Its name **decays** to a pointer to the first element, and `p + 1` moves one *element* (not one byte).



Adding 1 to an int pointer adds 4 bytes: the next element.

```
int a[3] = {10, 20, 30};
int* p = a;                  // the name decays to &a[0]
std::cout << *p;             // 10
std::cout << *(p + 1);       // 20
std::cout << p[2];           // 30: p[i] is exactly *(p + i)
```

The decay loses the size, which is the classic trap:

```
int a[5] = {1, 2, 3};        // fixed size; the other two elements are 0
a[10] = 7;                   // compiles, undefined behavior when it runs

void f(int arr[]) {          // really `int* arr`: the size is gone
    // the size of a POINTER (8), not of the array
    sizeof(arr);
}
char s[] = "hi";             // 3 chars: 'h', 'i' and the terminating '\0'
```

INSTEAD OF	USE	WHY
`int a[N]`	`std::array<int, N>`	knows its size, no decay, same speed
`new int[n]`	`std::vector<int>`	grows, frees itself, `.at(i)` checks bounds
`char*` strings	`std::string`	owns its memory, copies safely, has `.size()` and `.find()`
pointer plus a count	`std::span<int>` (C++20)	a non-owning view that remembers its size

---

References

A reference is not a second pointer. It is a **second name** for the same box, so using it needs no `*`.



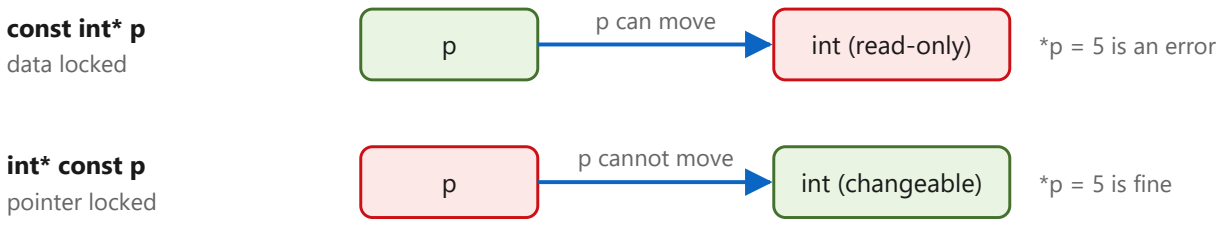
A pointer is a variable; a reference is another name.

```
int x = 1;
int& r = x;      // r is another name for x
r = 5;           // x is now 5
```

	POINTER `INT*`	REFERENCE `INT&`
Can be null	yes	no (must refer to something real)
Must be initialized	no (but should be)	yes, at declaration
Can be re-pointed	yes	no, bound for life
Syntax to use	`*p`, `p->member`	just use the name
Use for	optional things, arrays, ownership, C APIs	function parameters, aliases

---

`const` and pointers



Read right to left. Red = locked, green = free.

const before the star protects the data; after the star it protects the pointer.

`const` means "I will not modify this". With pointers, either the data or the pointer can be const. **Read the declaration right to left.**

```
int x = 1, y = 2;

const int* p1 = &x;          // "p1 is a pointer to const int"
// *p1 = 5;                  // ERROR: cannot change the int through p1
p1 = &y;                     // OK: the pointer itself may move

int* const p2 = &x;          // "p2 is a const pointer to int"
*p2 = 5;                     // OK: the int may change
// p2 = &y;                  // ERROR: the pointer may not move

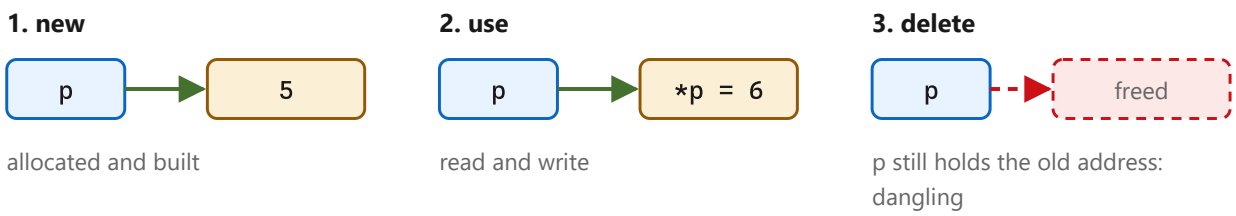
const int* const p3 = &x;    // neither can change
```

DECLARATION	READ AS	MOVE THE POINTER	CHANGE THE INT
`const int* p`	pointer to a const int	yes	no
`int* const p`	const pointer to an int	no	yes
`const int* const p`	const pointer to a const int	no	no

Make everything `const` that can be.

Dynamic memory: `new` and `delete`

`new` allocates on the heap **and** constructs the object; you must `delete` it, or it **leaks**. A heap object has three moments in its life, and the pointer outlives the last one.



delete frees the block but does not change p.

```cpp
// single object: new / delete
int*  one = new int(42);     delete one;
// array: new[] / delete[] (must match!)
int*  arr = new int[100];    delete[] arr;
```

BUG	WHAT HAPPENED
Memory leak	you lost the pointer (or forgot `delete`) so the memory is never freed
Dangling pointer	you used a pointer after the memory was freed
Double delete	you freed the same memory twice: undefined behavior, often a crash
Mismatch	`new[]` freed with `delete`, or the reverse

MODERN C++ Avoid raw `new` / `delete`: use `std::vector` and `std::unique_ptr` (Smart Pointers).

KEY POINT "What does `new` do beyond `malloc`?" It calls the constructor, throws `std::bad_alloc` on failure (instead of returning null), and returns a correctly typed pointer. `delete` calls the destructor and then frees the memory.

The classic memory bugs

Most "mysterious" crashes are one of these.

p = new int(1); p = new int(2);



Reassigning the only pointer to a block leaks the block.

BUG	EXAMPLE
Uninitialized read	`int n; use(n);`
Out-of-bounds	`int a[3]; a[3] = 1;` (valid indices are 0 to 2)
Null dereference	`int* p = nullptr; *p = 1;`
Use after free	`delete p; *p = 1;`
Leak	`p = new int(1); p = new int(2);` (the first block is lost)
Signed overflow	`INT_MAX + 1` (undefined for signed, wraps for unsigned)

Returning a reference to a local

```
int& bad() {
    int local = 42;
    return local;          // BUG: local is destroyed when bad() returns
}                          // the caller gets a dangling reference

int good() {
    int local = 42;
    return local;          // fine: returns a copy of the value
}
```

TOOLING Test with `-fsanitize=address,undefined`: it reports leaks, out-of-bounds, use-after-free and overflow with the exact line (see Tooling in the Basics lesson).

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Pointers from scratch	C++ Primer	2.3 Compound types	79 to 87
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463
Pointers and arrays	C++ Primer	3.2 Library string type	113 to 124
	C++ Primer	3.5 Arrays	142 to 153
	C++ Primer	3.6 Multidimensional arrays	154 to 161
	Programming: Principles and Practice	16.3 to 16.4 C-style strings; alternatives to pointer use	484 to 487
References	C++ Primer	2.3 Compound types	79 to 87
	Programming: Principles and Practice	16.1 to 16.2 Arrays, pointers and references	477 to 483
const and pointers	C++ Primer	2.4 const qualifier	88 to 95
Dynamic memory: new and delete	C++ Primer	12.2 Dynamic arrays	505 to 512
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463